

Ομαδική άσκηση: από την SQL στη MongoDB

- 1 Μελετήστε τη σελίδα του εγχειριδίου που αντιστοιχίζει εντολές της SQL σε εντολές της MongoDB.
- 2 Σε ομάδες, λύστε τα 13 ζητούμενα του φύλλου στη συλλογή books.
- 3 Παρουσιάστε τα αποτελέσματα στην ολομέλεια.

Όλες οι αίθουσες έχουν την ίδια άσκηση· ο καθένας δουλεύει στο δικό του mongosh.

Η σελίδα: SQL to MongoDB Mapping Chart

SQL, πίνακας people	MongoDB, συλλογή people
WHERE status = 'A' AND age = 50	{ status: "A", age: 50 }
WHERE status = 'A' OR age = 50	{ \$or: [{ status: "A" }, { age: 50 }] }
WHERE age > 25 AND age <= 50	{ age: { \$gt: 25, \$lte: 50 } }
WHERE user_id LIKE 'bc%'	{ user_id: /^bc/ }
SELECT user_id, status	{ user_id: 1, status: 1, _id: 0 }, δεύτερο όρισμα του find
ORDER BY user_id DESC	.sort({ user_id: -1 })
LIMIT 5 SKIP 10	.limit(5).skip(10)
SELECT COUNT(*) FROM people	db.people.countDocuments()

[Η σελίδα στο εγχειρίδιο.](#) Όπου γράφει `count()`, γράψτε `countDocuments()`.

Πώς δουλεύουμε

Ομάδες

Περίπου τέσσερα άτομα σε κάθε αίθουσα του Zoom· όλες οι αίθουσες έχουν το ίδιο φύλλο.

Εργαλείο

Ο καθένας στο δικό του mongosh, στη βάση library: `use library`.

Τι γράφετε

Για κάθε ζητούμενο, το ερώτημα της MongoDB και το πλήθος των εγγράφων που επιστρέφει.

Χρόνος

20 λεπτά στην αίθουσα· μετά δύο ομάδες παρουσιάζουν, 3 λεπτά η καθεμία.

Χωρίς τη συλλογή

Όποιος δεν ολοκλήρωσε την εισαγωγή, δουλεύει με την οθόνη ενός μέλους της ομάδας.

Ένα λυμένο παράδειγμα

SQL

```
SELECT title, pageCount FROM books
WHERE status = 'MEAP' AND pageCount > 0
ORDER BY pageCount DESC
LIMIT 3
```

MongoDB

```
db.books.find(
  { status: "MEAP", pageCount: { $gt: 0 } },
  { title: 1, pageCount: 1, _id: 0 }
).sort({ pageCount: -1 }).limit(3)
```

Αποτέλεσμα

```
[
  { title: 'Groovy in Action, Second Edition', pageCount: 700 },
  { title: 'F# in Action', pageCount: 425 },
  { title: 'Distributed Agile in Action', pageCount: 325 }
]
```

- `WHERE` γίνεται φίλτρο: το `AND` είναι το κόμμα.
- `SELECT title, pageCount` γίνεται προβολή.
- `ORDER BY ... DESC` γίνεται `sort` με -1· το `LIMIT` γίνεται `limit`.

Στις αίθουσες

Σε κάθε αίθουσα, με το φύλλο της άσκησης:

- 1 Ο καθένας στο δικό του mongosh: `use library`.
- 2 Για κάθε ζητούμενο, το ερώτημα της MongoDB και το πλήθος, με `countDocuments` και το ίδιο φίλτρο.
- 3 Ένα μέλος κρατά τις απαντήσεις της ομάδας.

Βοήθημα: [SQL to MongoDB Mapping Chart](#), στο εγχειρίδιο της MongoDB.

20:00

Δύο ομάδες παρουσιάζουν

Δύο ομάδες, τρία λεπτά η καθεμία:

- 1 Η πρώτη ομάδα: τα ζητούμενα 4 έως 8, με το ερώτημα και το πλήθος.
- 2 Η δεύτερη ομάδα: τα ζητούμενα 9 έως 13, με τον ίδιο τρόπο.
- 3 Οι υπόλοιπες ομάδες συγκρίνουν με τα δικά τους πλήθη.

03:00

Java: 96, όχι 76 ούτε 97

Εντολές

```
db.books.countDocuments({ categories: "Java" })
db.books.countDocuments({ categories: ["Java"] })
db.books.countDocuments({ categories: "java" })
db.books.countDocuments({ categories: { $in: ["Java", "java"] } })
```

Αποτελέσματα

```
96
76
1
97
```

- Ζητούμενο 4 (πρώτη εντολή): 96 βιβλία.
- Με αγκύλες, 76: μόνο όσα έχουν ως μοναδική κατηγορία τη Java.
- Ένα βιβλίο έχει «java» με πεζά. Αν το θέλουμε κι αυτό, `$in` και τα δύο: 97.

AND ή OR

Εντολές

```
db.books.countDocuments({ categories: "Java", status: "PUBLISH" })
db.books.countDocuments({ $or: [ { categories: "Java" },
                                   { status: "PUBLISH" } ] })
db.books.countDocuments({ status: "PUBLISH" })
db.books.countDocuments({ categories: "Java", status: "MEAP" })
```

Αποτελέσματα

```
95
364
363
1
```

- Ζητούμενο 5, AND: 95 βιβλία Java που κυκλοφόρησαν.
- Ζητούμενο 6, OR: 364, δηλαδή τα 363 που κυκλοφόρησαν και ένα βιβλίο Java σε MEAP.
- Το «είτε... είτε» της εκφώνησης είναι OR: αρκεί μία από τις δύο συνθήκες.

Ταξινόμηση κειμένου: τα πεζά στο τέλος

Εντολές

```
db.books.find({ categories: "Java" }, { title: 1, _id: 0 })
  .sort({ title: -1 }).limit(3)
db.books.find({ categories: "Java" }, { title: 1, _id: 0 })
  .sort({ title: -1 }).collation({ locale: "en" }).limit(3)
```

Αποτελέσματα

```
[
  { title: 'jQuery in Action, Second Edition' },
  { title: 'iText in Action, Second Edition' },
  { title: 'XML Parsing with Digester' }
]
[
  { title: 'XML Parsing with Digester' },
  { title: 'XDoclet in Action' },
  { title: 'Working with the Logging and Discovery Components' }
]
```

- Η MongoDB συγκρίνει τα κείμενα χαρακτήρα προς χαρακτήρα: όλα τα κεφαλαία πριν από τα πεζά. Το «jQuery» έρχεται μετά το «XML».
- Με `collation` και `locale: "en"`, η σειρά γίνεται αλφαβητική, ανεξάρτητα από πεζά και κεφαλαία.
- Ζητούμενο 9: στο τέλος της λίστας είναι τα 19 βιβλία Java με 0 σελίδες.

Κείμενο: λέξη ή κομμάτι λέξης

Εντολές

```
db.books.countDocuments({ title: /Java/ })
db.books.countDocuments({ title: /\bJava\b/ })
db.books.countDocuments({ title: /^Java/ })
db.books.countDocuments({ title: /^Java\b/ })
```

Αποτελέσματα

```
38
29
14
11
```

- Ζητούμενο 11: το `/Java/` βρίσκει 38 τίτλους, μαζί με JavaScript, JavaServer, JavaFX. Μόνο η λέξη Java: `\b` πριν και μετά, 29.
- Ζητούμενο 12: το `/^Java/` βρίσκει 14· οι 3 από αυτούς αρχίζουν με JavaServer, JavaFX, JavaScript.
- Ζητούμενο 13: `/^\.a\.a/`, 19 τίτλοι· η `.` της regex αντιστοιχεί στο `_` του LIKE της SQL.

Τρία σημεία από το Μέρος 2

- 1 SQL και MongoDB λένε το ίδιο με άλλη σύνταξη: το WHERE γίνεται φίλτρο, το SELECT προβολή, το ORDER BY sort, το LIMIT limit.
- 2 Όπου η SQL θα είχε μία τιμή, η MongoDB μπορεί να έχει λίστα: εκεί η ισότητα σημαίνει «περιέχει».
- 3 Τα κείμενα συγκρίνονται χαρακτήρα προς χαρακτήρα: Java, java και JavaScript δίνουν τρία διαφορετικά αποτελέσματα.

Διάλειμμα

Επιστρέφουμε σε **15'**

15:00

Στο Μέρος 3: ενημέρωση, διαγραφή και συνάθροιση, και η Εργασία 2 ενότητα προς ενότητα.